Group Presentation

AI help analyze data: A case study of COVID19 prediction

Ziheng LYU Zeming CHEN

Jiaming LIU Xin WU Ang MA

Faculty of Business and Management

BNU-HKBU United International College

Nov 30, 2024

Table of Contents

Introduction (Presenter: Ang MA)

Machine Learning Model (Presenter: Xin WU)

Parameter Adjustment (Presenter: Jiaming LIU)

Multi-stage Decision System (Presenter: Zeming CHEN)

Deep Learning Method (Presenter: Ziheng LYU)

Artificial Intelligence and Deep Learning

Introduction (Presenter: Ang MA)

Introduction

Background

2020 Feb epidemic, the shortage of medical resources cause a great problem of Misdiagnosis and delayed treatment.

Purpose

We've decided to create a method to solving this issue for future occurrence.

Data

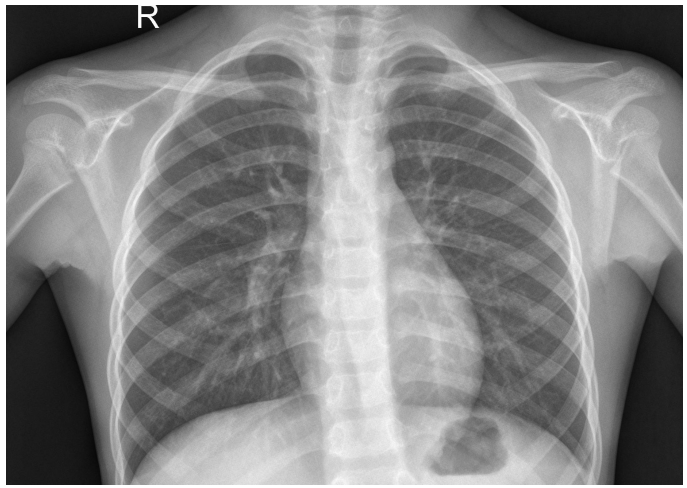
- CSV file from Kaggle, contain the symptoms during 2020 Feb
- CT Images from Kaggle during 2020 Feb

Sample Date

CSV File

Breathing Pr	Fever	Dry Cough	Sore throat	Running Nos	Asthma	Chronic Lung	Headache
Yes	Yes	Yes	Yes	Yes	No	No	No
Yes	Yes	Yes	Yes	No	Yes	Yes	Yes
Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Yes	Yes	Yes	No	No	Yes	No	No
Yes	Yes	Yes	Yes	Yes	No	Yes	Yes
Yes	Yes	Yes	No	No	No	No	No
Yes	Yes	Yes	No	No	No	Yes	No
Yes	Yes	Yes	No	Yes	Yes	No	No
Yes	Yes	Yes	No	Yes	No	Yes	No
Yes	Yes	Yes	No	No	Yes	No	No
Yes	Yes	Yes	No	No	No	Yes	No
Yes	Yes	Yes	Yes	Yes	Yes	No	No

CT Image



Artificial Intelligence and Deep Learning

Machine Learning Model (Presenter: Xin WU)

Naive Bayes

Goal

Compare posterior probability, determine labeled class

Bayes Theorem

$$P\left(y_k|x\right) = \frac{P\left(x|y_k\right)P\left(y_k\right)}{\sum_k P\left(x|y_k\right)P\left(y_k\right)} \tag{1}$$

Posterior Probability

$Y = 0$ if

$$P\left(Y = 0|X\right) > P\left(Y = 1|X\right)$$

$Y = 1$ if

$$P\left(Y = 0|X\right) < P\left(Y = 1|X\right)$$

If X is conditionally independent of Y:

Conditional Probability

$$P\left(x|y_k\right) = P\left(x_1, x_2, \ldots, x_n|y_k\right) = \prod_{i=1}^{n} P\left(x_i|y_k\right) \tag{2}$$

Naive Bayes Classifier

$$\max\left[\frac{P\left(y_k\right)\prod_{i=1}^{n} P\left(x_i|y_k\right)}{\sum_k P\left(y_k\right)\prod_{i=1}^{n} P\left(x_i|y_k\right)}\right] \Rightarrow \arg\max_{y_k} P\left(y_k\right)\prod_{i=1}^{n} P\left(x_i|y_k\right) \tag{3}$$

Logistic Regression

Goal

Construct Linear Function with Sigmoid(σ) to predict classification

Linear Function

$$z = \mathbf{w}^T \mathbf{x} + b$$

Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x} - b}} \tag{4}$$

Therefore,

$$P(Y = 1|X) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x} + b}} \quad P(Y = 0|X) = \frac{1}{1 + e^{\mathbf{w}^T \mathbf{x} + b}} \tag{5}$$

Construct Logistic Regression

Training Data

$$\left\{\left(X^{(j)}, Y^{(j)}\right)\right\}_{j=1}^{n} \quad X^{(j)} = \left(X_1^{(j)}, \ldots, X_d^{(j)}\right)$$

Maximum Likelihood Estimates

$$\hat{w}_{\mathrm{MLE}} = \arg\max_w l(w) = \arg\max_w \prod_{j=1}^{n} P\left(X^{(j)}, Y^{(j)} \middle| w\right) \tag{6}$$

Logarithm Process

$$\ln l\left(w\right) = \sum_j \left\{ y^j \left(\sum_{i=1}^{d} w_i x_i^j \right) - \ln \left[1 + \exp\left(w_0 + \sum_{i=1}^{d} w_i x_i^j \right) \right] \right\} \tag{7}$$

Gradient Descent

Get optimal coefficient $\mathbf{w}$.

$$\mathbf{w}_i^{(t+1)} = \mathbf{w}_i^{(t)} + \eta \frac{\partial l\left(\mathbf{w}\right)}{\partial \mathbf{w}_i}\bigg|_t \tag{8}$$

Linear Function

Build z, combine the Sigmoid Function, classify Y (0 or 1).

$$\begin{aligned} P\left(Y=0|X\right) > P(Y=1|X) \quad &\Rightarrow \quad Y=0 \\ P\left(Y=0|X\right) < P(Y=1|X) \quad &\Rightarrow \quad Y=1 \end{aligned}$$

Decision Tree

Goal

Build optimal Decision Tree allows to classify Y.

Optimal Decision Tree

Maximum purity of nodes is the minimum information Entropy(S)

$$E\left(p_{\oplus}, p_{\ominus}\right) = H\left(S\right) \equiv -p_{\oplus}\log_2 p_{\oplus} - p_{\ominus}\log_2 p_{\ominus} \tag{9}$$

Step 1

Selecting features with the highest Information Gain as its parent node and child node

$$\text{Gain}\left(S, A\right) = \text{Entropy}\left(S\right)\sum_{v\in A}\frac{|S_v|}{|S|}\cdot\text{Entropy}\left(S_v\right) \tag{10}$$

Step 2

Cycle step 1 until reaching the predefined depth or the information entropy of a node is 0, then stop.

SVM

Goal

Find the best hyperplane to sepreate data

Best Hyperplane

- Constantly update coefficient w
- Let distance between supporting vector reach maximum

Hard-margin SVM

Hyperplane: $\mathbf{w}^T X + b = 0$

Former Problem

Input:

$S = \{(x_1, y_1), \ldots, (x_m, y_m)\}$

Maximize γ under the constraint:

- $||w||^2 = 1$
- $y_i w \cdot x_i \geq \gamma, \quad \forall i$

New Problem ($w' \leftarrow w/\gamma$)

Input:

$S = \{(x_1, y_1), \ldots, (x_m, y_m)\}$

Minimize w' under the constraint:

- $||w||^2 = 1$
- $y_i w' \cdot x_i \geq 1, \quad \forall i$

Use Lagrangian and Dual, converts primal form to dual problem

$$\max_{\alpha}(L) = \max_{\alpha}\left[-\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j y_i y_j X_i^T X_j + \sum_{i=1}^{n}\alpha_i\right] \tag{11}$$

where $\alpha \geq 0$, $i = 1, 2, \ldots, n$.

Final Classifier

$$W = \sum_{i=1}^{n}\alpha_i y_i X_i \tag{12}$$

Method 1: Soft-Margin SVM

Input

$$S = \{(x_1, y_1), \ldots, (x_m, y_m)\}$$

Objective

$$\arg \min_{w,\xi_1,\ldots,\xi_m} ||w||^2 + C\sigma_i\xi_i$$

Subject to

$$y_i w \cdot x_i \geq 1 - \xi$$

Lagrangian Transform

Objective

$$\arg\min_{\alpha} \frac{1}{2} \sum_{i} y_i y_j \alpha_i \alpha_j x_i \cdot x_j - \sum_{i} \alpha_i$$

Subject to

$$0 \leq \alpha_i \leq C$$

$$\sum_{i} y_i \alpha_i = 0$$

Final Classifier

$$w = \sigma_i \alpha_i y_i x_i$$

Method 2: Kernel Function

Objective

$$\min_{W,\xi} f(W,\xi) = \frac{1}{2}||W||^2 + C\sum_{i=1}^{m}\xi_i$$

Subject to

$$g_i(W,b) = 1 - y_i\left(W^T X_i + B\right) - \xi_i \leq 0$$
$$h_i(\xi_i) = -\xi_i \leq 0$$

Lagrangian Transform

Objective

$$\max\{L(\alpha)\} = -\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j y_i y_j \phi^T(X_i)\,\phi(X_j) + \sum_{i=1}^{n}\alpha_i$$

Subject to

$$0 \leq \alpha_i \leq C \quad \sum_{i=1}^{n}\alpha_i y_i = 0$$

Final Classifier

$$W = \sum_{i=1}^{n}\alpha_i y_i X_i \quad f(X) = \sum_{i=1}^{n}\alpha_i y_i K(X, X_i) + b$$

$$K\left(X_1, X_2\right) = \phi^T\left(X_1\right)\phi\left(X_2\right)$$

Linear Model

$$K(X, Z) = X^T Z + C$$

Polynomial Kernel

$$K\left(X, Z\right) = \left(X^T Z\right)^d + C$$

Sigmoid

$$K\left(X, Z\right) = \tanh\left(\beta X^T Z + 0\right)$$

Laplacian Kernel

$$K\left(X_i, X_j\right) = \exp\left(-\frac{||x_i - x_j||}{\sigma}\right)$$

Gaussian Kernel

$$K(X, Z) = \exp\left(-\frac{||X - Z||^2}{2\sigma^2}\right)$$

Artificial Intelligence and Deep Learning

Parameter Adjustment (Presenter: Jiaming LIU)

Scores

- TP (True Positives): Correctly predicted positive samples
- TN (True Negatives): Correctly predicted negative samples
- FP (False Positive): Incorrectly predicted positive samples
- FN (False Negatives): Incorrectly predicted negative samples

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{13}$$

Precision

$$\text{Precision} = \frac{TP}{TP + FP} \tag{14}$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN} \tag{15}$$

F1-score

$$\text{F1} - \text{score} = \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{16}$$

Naive Bayes

Parameters

- Alpha: Default is 1 which is for the dataset.
- Binarize: The params that set data to 0 and 1 (no need).

Solver

- Gaussian Bayes: for continuous variables
- Bernoulli Bayes: for discrete variables (more suitable)

Before Adjustment

```
Average Accuracy: 0.7604
Average Precision: 1.0000
Average Recall: 0.7027
Average F1-Score: 0.8251
```

After Adjustment

```
Average Accuracy: 0.9656
Average Precision: 0.9713
Average Recall: 0.9865
Average F1-Score: 0.9788
```

Logistic Regression

Parameters

- C: Inverse of regularization strength $(1/\lambda)$
- Solver: Methods to find optimization weight (w)
- Regularization type ($L_1 : R(w) = \lambda \sum |w_i|$, $L_2 : R(w) = \lambda \sum w_i^2$)

Changes

- C: $1 \rightarrow 0.992$
- Solver: `liblinear` $\rightarrow$ `lbfgs`
- Regularization Type: $L_1 \rightarrow L_2$

Before Adjustment

```
Average Accuracy: 0.9679
Average Precision: 0.9732
Average Recall: 0.9874
Average F1-Score: 0.9802
```

After Adjustment

```
Average Accuracy: 0.9687
Average Precision: 0.9760
Average Recall: 0.9855
Average F1-Score: 0.9807
```

Decision Tree

Parameters

- `min_samples_split`: minimum number of samples required to split an internal node
- `min_samples_leaf`: the minimum number of samples required to be at a leaf node
- `max_depth`: maximum depth of the decision tree
- `max_features`: the maximum number of features to consider (e.g. `"auto"`, `"sqrt"`, `"log2"`)

Adjustment

During adjustment, the default highly matches the dataset, no change of params in this model.

Evaluation

```
Average Accuracy: 0.9820    Average Precision: 0.9933
Average Recall: 0.9843      Average F1-Score: 0.9888
```

SVM

Kernal

- `"rbf"`: suitable for multi-dimensional situation/complex data
- `"linear"`: suitable for linear situation(simple data)

Evaluation

Choose `"rbf"`: Increase dimension to guarantee the creditability of model

For rbf Method

C Parameter (Tolerance)

Smaller C values indicate lower tolerance (overfitting); Larger C values indicate higher tolerance (underfitting).

Gamma Parameter (Kernel Coefficient)

Larger gamma values lead to more complex boundaries and potential overfitting, smaller values result in simpler boundaries and potential underfitting.

Adjustment

- C: $1 \rightarrow 8$
- Gamma: $1/\text{num of features}\ (1/20) \rightarrow 0.1$

Before Adjustment

```
Average Accuracy: 0.9674
Average Precision: 0.9663
Average Recall: 0.9945
Average F1-Score: 0.9801
```

After Adjustment

```
Average Accuracy: 0.9806
Average Precision: 0.9884
Average Recall: 0.9876
Average F1-Score: 0.9880
```

ROC Curve

- X-axis: False Positive Rate (FPR)
- Y-axis: True Positive Rate (TPR)
- Curve: ROC
- Fields: AUC

Comprehensive view of a binary classification models' performance, illustrating the trade-off between true positive and the false positive rate.

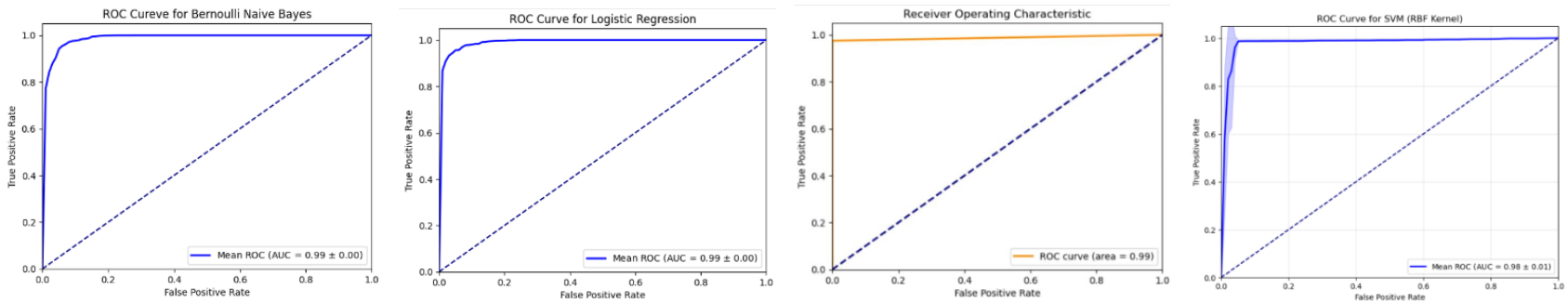


Figure: ROC Curve

Artificial Intelligence and Deep Learning

Multi-stage Decision System (Presenter: Zeming CHEN)

General Description

Reason

- Limited medical resources
- Multi-stage decision system

How

- MPGA to combine previous models
- Assign priority

MPGA Modeling

Bernoulli Naive Bayes

Performs well in simple scenarios (e.g., quick screening) but may struggle with complex data.

Decision Tree

Suitable for medium-sized datasets, useful for analyzing key influencing factors (e.g., age, health conditions)

Logistic Regression

Ideal for identifying key risk factors (e.g., high risk in elderly or patients with chronic diseases).

SVM

Excels in precise predictions for multi-dimensional features (e.g., medical resource distribution, infection trends).

Matrix and requirement for each item:

Decision Weight Matrix

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \cdots & \cdots & \ddots & \cdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

where $a_{ij} > 0$, $a_{ij} = 1/a_{ji}$, $a_{ii} = 1$.

Scale	Definition
1	Equal Importance
3	Moderate Importance
5	Strong Importance
7	Very Strong and Proven Importance
9	Absolute Importance
$2, 4, 6, 8$	Intermediate Values between the two adjacent judgments

Table: Saatty Scale Table

Fill the scale into the matrix and obtain this:

Decision Weight Matirx

	NB	DT	LR	SVM
Naive Bayes	1	$1/3$	$1/5$	$1/7$
Decision Tree	3	1	$1/3$	$1/5$
Logistic Regression	5	3	1	$1/3$
SVM	7	5	3	1

Model Weight Calculation

n-dimension vectors

$$\bar{w}_i = \sqrt{\prod_{j=1}^{n} a_{ij}} \quad i, j = 1, 2, \cdots, n \tag{17}$$

Weight Vectors

$$w_i = \frac{\bar{w}_i}{\sum_{j=1}^{m} w_j} \tag{18}$$

Lambda (λ)

$$\lambda_{\max} = \frac{1}{n} \sum_{i=1}^{n} \frac{(\mathbf{A}w)_i}{w_i} \tag{19}$$

Verification

Utlize Consistency Ratio Test

$$\mathrm{CR} = \mathrm{CI}/\mathrm{RI} \tag{20}$$

RI can be found in the Random Consistency Index (RCI)

n	1	2	3	4	5	6	7	8	9
RI	0	0	.58	.90	1.12	1.24	1.32	1.41	1.45

Table: Random Consistency Index (RCI)

CI formula

$$\mathrm{CI} = \frac{\lambda_{\max} - n}{n - 1} \tag{21}$$

MPGA Formula

Because

$$\mathrm{CR} = .04 < .1$$

Accept and weights

$$\begin{aligned} w_{m_1} &= 0.0550 \quad \text{(Navie Bayes)} \\ w_{m_2} &= 0.1178 \quad \text{(Decision Tree)} \\ w_{m_3} &= 0.2634 \quad \text{(Logistic Regression)} \\ w_{m_4} &= 0.5638 \quad \text{(SVM)} \end{aligned}$$

Built MPGA with this equation

$$\mathrm{MPGA} = w_{m_1} m_1 + w_{m_2} m_2 + w_{m_3} m_3 + w_{m_4} m_4 \qquad (22)$$

And we obtain

$$\mathrm{MPGA} = 0.055 m_1 + 0.1178 m_2 + 0.2634 m_3 + 0.5638 m_4$$

Set Thresholds

```
thresholds = np.arange(0, 1.01, 0.01)
best_threshold = 0
best_f1 = 0
f1_scores = []
for t in thresholds:
    final_predictions = (weighted_probs >= t).astype(int)
    f1 = f1_score(y_test, final_predictions)
    f1_scores.append(f1)
    if f1 > best_f1:
        best_f1 = f1
        best_threshold = t
print(f"Best threshold: {best_threshold}")
print(f"Corresponding F1 Score: {best_f1:.4f}")
```

$$\max(F_1) \rightarrow \text{thresholds} \rightarrow 0.31$$

Which means that we can conclude COVID-19 affection if $F_1 > .31$

Artificial Intelligence and Deep Learning

Deep Learning Method (Presenter: Ziheng LYU)

Convolutional Neural Network

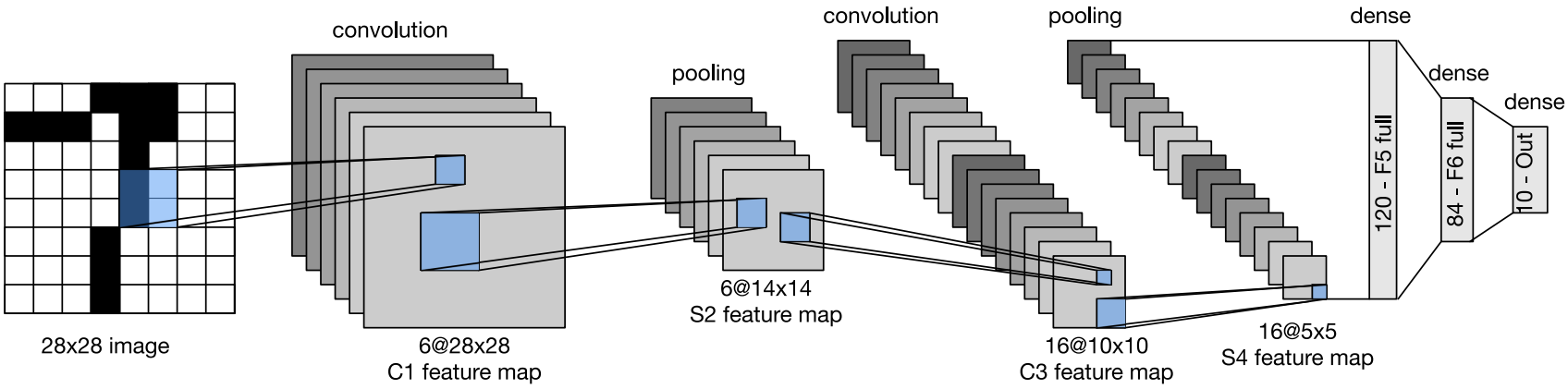


Figure: LeNet-5 Architecture

CNN Structure

- Convolutional Layer
- Pooling Layer
- Fully Connected Layer

Initial Step

Transform the images:

- Import data
- Resize (sub-sampling)
- Image enhancement
- Rotate image
- To tensor
- Normalize

Then start CNN:

- Convolution
- Pooling
- Convolution
- Pooling
- Fully Connected
- Fully Connected

Convolution

A convolution layer is made of several convolution filters (kernels).

$$(\mathbf{x} * \mathbf{f})(i,j,c) = \varphi\left(\sum_{k=-K}^{K}\sum_{l=-L}^{L}\sum_{c'} f^{c}_{k,l,c'} \cdot x_{i+k,j+l,c'} + b_c\right)$$

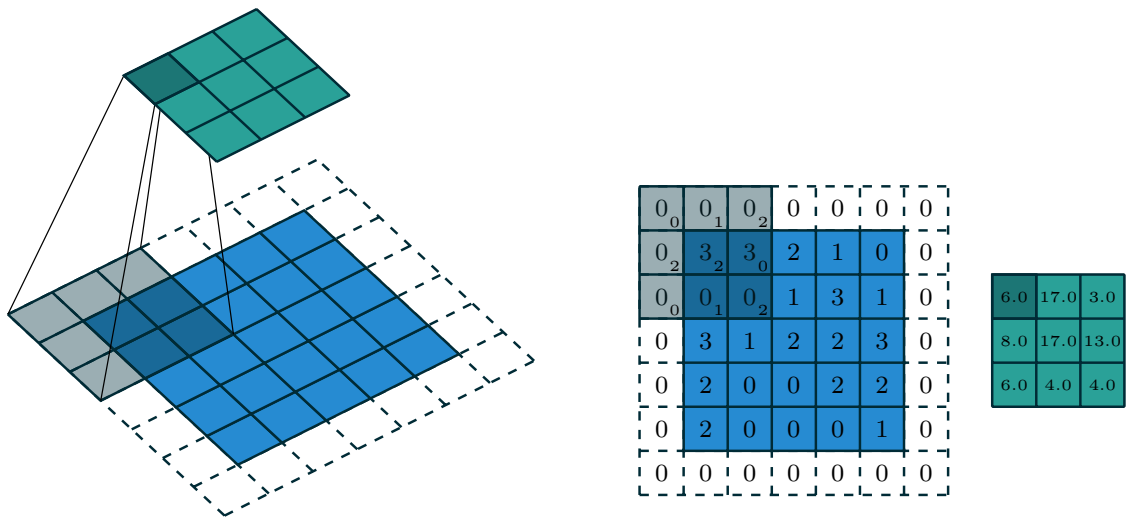


Figure: Convolution Layer

Pooling

Max-pooling to reduce the spatial dimensions of feature maps.

$$\text{MaxPool}\left(x\,|f,s\right)_{i,j,c} = \max\left(x_{is_H:is_H+f_H-1,js_W:js_W+f_W-1,c}\right)$$

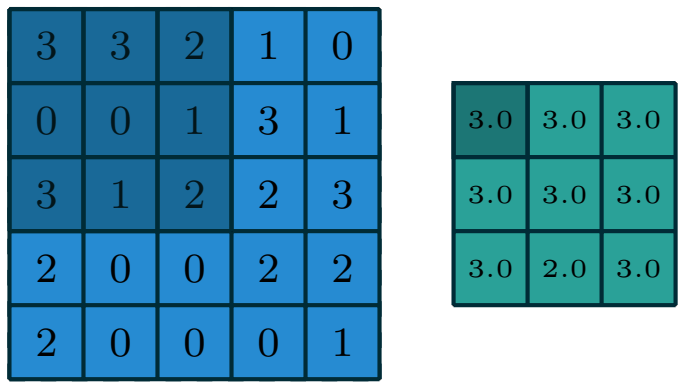
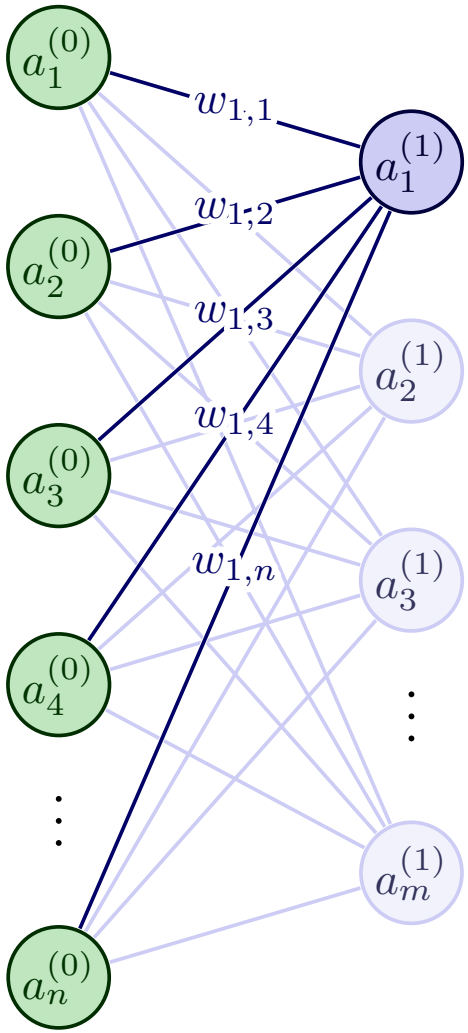


Figure: Maximum Pooling Layer

Activation



$$= \sigma\left(w_{1,1}a_1^{(0)} + w_{1,2}a_2^{(0)} + \ldots + w_{1,n}a_n^{(0)} + b_1^{(0)}\right)$$

$$= \sigma\left(\sum_{i=1}^{n} w_{1,i}a_i^{(0)} + b_1^{(0)}\right) \quad \text{where} \quad \sigma = \frac{1}{1+\exp(-z)}$$

$$\begin{pmatrix} a_1^{(1)} \\ a_2^{(1)} \\ \vdots \\ a_m^{(1)} \end{pmatrix} = \sigma\left[\begin{pmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,n} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,1} & w_{m,2} & \cdots & w_{m,n} \end{pmatrix}\begin{pmatrix} a_1^{(0)} \\ a_2^{(0)} \\ \vdots \\ a_n^{(0)} \end{pmatrix} + \begin{pmatrix} b_1^{(0)} \\ b_2^{(0)} \\ \vdots \\ b_m^{(0)} \end{pmatrix}\right]$$

$$\mathbf{a}^{(1)} = \sigma\left(\mathbf{W}^{(0)}\mathbf{a}^{(0)} + \mathbf{b}^{(0)}\right)$$

Figure: Fully connected (FC) Layer

Structure/Output

Pseudo-Strcture

```
torch.nn.functional.relu(self.nn.Conv2d(x))
torch.nn.functional.max_pool2d(x)
torch.nn.functional.relu(self.nn.Conv2d(x))
torch.nn.functional.max_pool2d(x)
torch.nn.functional.sigmoid(nn.Linear(x))
torch.nn.functional.sigmoid(nn.Linear(x))
```

Output

```
Results per fold:
Fold 0 - Loss: 0.0620, Accuracy: 97.92%
Fold 1 - Loss: 0.0521, Accuracy: 98.38%
Fold 2 - Loss: 0.0493, Accuracy: 98.38%
Fold 3 - Loss: 0.0623, Accuracy: 98.15%
Fold 4 - Loss: 0.1556, Accuracy: 96.75%
Average Validation Accuracy: 97.92%
```